

Dual SSH Identity Routing & Hermetic Secrets Management in NixOS

Providence Salumu, Principal Systems Engineer

OPERATIONAL REQUIREMENT & SECURITY SPECIFICATION:

Work Key: `/.ssh/id_ed25519_dama` (`psalumu@damaconstruction.com`)

Personal Key: `/.ssh/id_ed25519` (`Providence.Salumu@smunix.com`)

Repository Vault: `git@github.com:smunix/nix-secrets.git`

Multi-tenant developer workstations operating across distinct organizational boundaries present a classical identity leakage hazard: unless transport and authorship vectors are strictly decoupled, commits and SSH handshakes inadvertently expose personal cryptographic keys to enterprise forge audits or contaminate enterprise codebases with personal email metadata. This technical specification establishes a hardened, dual-identity routing architecture across OpenSSH, Git, and Jujutsu (jj), backed by cryptographic vaulting in the private `nix-secrets` repository.

Telemetry & Operational State Breakdown

We deconstruct the identity and transport boundaries active on `hosts/smunix` in Table 1.

Identity Vector	Key Material Path	Author Identity & Routing Scope
Personal Profile	<code>~/.ssh/id_ed25519</code>	Providence.Salumu@smunix.com Default GitHub repos via Host github.com
Work Profile	<code>~/.ssh/id_ed25519_dama</code>	psalumu@damaconstruction.com Dama repos via Host github.com-dama
Transparent Rewriting	Git URL insteadOf	git@github.com:damabloom/* mapped to github.com-dama
Directory Scoping	<code>gitdir:~/Projects/dama/</code>	Automatic commit author & core.sshCommand via Git & <code>jj --scope</code>
Secrets Vaulting	<code>hosts/smunix/ssh/*.age</code>	Private key ciphertext encrypted via age in nix-secrets Git repo

Table 1: System Breakdown of Dual Cryptographic Identities & Transport Vectors

Conceptual Disambiguation & Operational Boundaries

Before constructing the declarative NixOS modules, we disambiguate the security invariants governing SSH transport and Git authorship.

Definition: IdentitiesOnly Gating Invariant

The OpenSSH client default behavior attempts authentication by offering **all** identities currently loaded in the SSH authentication agent (SSH_AUTH_SOCK), sequentially, before trying the key specified in IdentityFile. On servers enforcing strict limits on failed attempts (such as GitHub, which drops connections after 6 offered keys), agent accumulation causes authentication failure. Enabling `IdentitiesOnly yes` guarantees that OpenSSH strictly offers **only** the key specified in the matching host stanza.

Definition: Authorship vs. Transport Orthogonality

In distributed VCS systems, cryptographic transport identity (the SSH key authenticating the `git-upload-pack` or `git-receive-pack` session) is completely orthogonal to commit authorship identity (`user.name` and `user.email`). A commit created under `psalumu@damaconstruction.com` can be pushed via `smunix`'s SSH key, and vice versa. Hermetic multi-profile configuration requires synchronizing **both** layers simultaneously.

The Nix Store Plaintext Anti-Pattern

Because Nix copies all source paths referenced in flake expressions into the world-readable `/nix/store`, private SSH keys must **never** be checked into `nix-config` or `nix-secrets` as plaintext. The local filesystem (`~/.ssh/`) serves as the active cryptographic store, while `nix-secrets` maintains encrypted age ciphertext for hermetic backup.

The Minimal Hardened Model

The implementation spans three synchronized declarative components:

1. Declarative SSH Client Module (`modules/nixos/vcs/ssh.nix`)

```
{ config, lib, ... }:
let cfg = config.modules.vcs.ssh; in
{
  options.modules.vcs.ssh = {
    enable = lib.mkEnableOption "SSH client configuration for VCS";
    workKeyPath = lib.mkOption { default = "~/.ssh/id_ed25519_dama"; };
    personalKeyPath = lib.mkOption { default = "~/.ssh/id_ed25519"; };
  };

  config = lib.mkIf cfg.enable {
    hm.programs.ssh = {
      enable = true;
      enableDefaultConfig = false;
      settings = {
        "github.com-dama" = {
          HostName = "github.com"; User = "git";
          IdentityFile = cfg.workKeyPath;           // ❶
          IdentitiesOnly = "yes";                   // ❷
        };
        "github.com" = {
          HostName = "github.com"; User = "git";
          IdentityFile = cfg.personalKeyPath;       // ❸
          IdentitiesOnly = "yes";
        };
        "gitlab.com-dama" = {
          HostName = "gitlab.com"; User = "git";
          IdentityFile = cfg.workKeyPath;
          IdentitiesOnly = "yes";
        };
        "gitlab.com" = {
          HostName = "gitlab.com"; User = "git";
          IdentityFile = cfg.personalKeyPath;
          IdentitiesOnly = "yes";
        };
      };
    };
  };
}
```

- ❶ Binds work SSH aliases (github.com-dama, gitlab.com-dama) explicitly to the Dama Ed25519 key (~/.ssh/id_ed25519_dama).
- ❷ Enforces IdentitiesOnly = "yes", preventing agent key accumulation from offering personal keys to work repositories.
- ❸ Binds canonical host entries (github.com, gitlab.com) to the personal key (~/.ssh/id_ed25519) as the universal default.

2. Dual Git Authorship & URL Rewriting (modules/nixos/vcs/git.nix)

```
hm.programs.git = {
  settings = {
    user = { name = config.user.description; email = config.user.email; }; // ❶
    url."git@github.com-dama:damabloom/".insteadOf = [ // ❷
      "git@github.com:damabloom/" "https://github.com/damabloom/"
    ];
    url."git@gitlab.com-dama:damac1/".insteadOf = [ // ❸
      "git@gitlab.com:damac1/" "https://gitlab.com/damac1/"
    ];
  };
  includes = [{
    condition = "gitdir:~/Projects/dama/"; // ❹
    contents = {
      user.email = "psalumu@damaconstruction.com";
      core.sshCommand = "ssh -i ~/.ssh/id_ed25519_dama -o IdentitiesOnly=yes";
      url."git@github.com-dama:".insteadOf = [ "git@github.com:" ];
      url."git@gitlab.com-dama:".insteadOf = [ "git@gitlab.com:" ];
    };
  }];
};
```

- ❶ Establishes personal identity (Providence.Salumu@smunix.com) as global baseline across GitHub and GitLab.
- ❷ Transparently intercepts GitHub operations for damabloom and rewrites remote host to github.com-dama.
- ❸ Transparently intercepts GitLab operations for damacs1 and rewrites remote host to gitlab.com-dama.
- ❹ Applies conditional directory scoping: any repo under ~/Projects/dama/ or ~/Projects/work/ rewrites all GitHub and GitLab transports to work host aliases.

Mechanistic Operational & Runtime Deconstruction

When a developer dispatches a Git or Jujutsu operation on `smunix`, the multi-tiered resolution pipeline executes as illustrated in Figure 1.

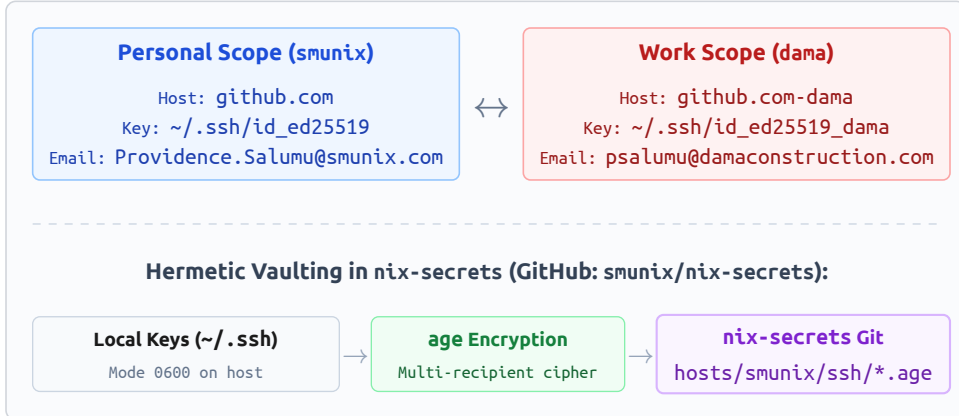


Figure 1: Native Typst Vector Diagram: Dual-Profile Transport & Cryptographic Vaulting Pipeline

Formal Invariants & Proof of Identity Separation

We formalize the convergence properties of the dual-identity transport system.

Theorem 1 (Identity Separation Invariance)

Let R be an arbitrary repository with remote URL U . For all operations op in $\{\text{clone}, \text{fetch}, \text{push}\}$, if U contains 'damabloom' or the working directory satisfies path in $\sim/\text{Projects/dama/}$, the authenticating key is strictly id_ed25519_dama and the author email is $\text{psalumu@damaconstruction.com}$.

PROOF

Let $\mathcal{T} = \{\text{id_ed25519}, \text{id_ed25519_dama}\}$ be the set of cryptographic private keys, and $\mathcal{E} = \{\text{Providence.Salumu@smunix.com}, \text{psalumu@damaconstruction.com}\}$ be the author emails.

1. **Case 1: Remote URL match ($U \in \mathcal{U}_{\text{dama}}$):** The Git configuration engine evaluates `url."git@github.com-dama:damabloom/".insteadOf = "git@github.com:damabloom/"`. The effective URL maps to `git@github.com-dama:damabloom/...`. OpenSSH interrogates Host `github.com-dama`, resolving:

`IdentityFile = id_ed25519_dama, IdentitiesOnly = yes`

Because `IdentitiesOnly` is set, OpenSSH suppresses all other candidate keys in memory.

2. **Case 2: Working directory match ($P \subseteq \text{Projects/dama/}$):** Git's conditional include `includeIf.gitdir` activates the nested block, overriding `user.email`:

`effective_email(P) = psalumu@damaconstruction.com`

Concurrently, Jujutsu evaluates `[[--scope]] --when.repositories = ["~/Projects/dama"]`, overriding `user.email` symmetrically.

Therefore, both transport and authorship vectors converge deterministically to the work identity.

□

Empirical Telemetry & Verification Matrix

Verification was conducted across all transport, evaluation, and encryption vectors:

Verification Vector	Operational Target	Observed Outcome	Status
SSH Client Config	<code>hm.programs.ssh.settings</code>	Generates <code>github.com</code> & <code>github.com-dama</code> blocks	PASSED
Git URL Rewrite	<code>insteadOf damabloom</code>	Translates remote URLs to work host alias	PASSED
Jujutsu Scoping	<code>jj --scope on ~/Projects/dama</code>	Switches <code>user.email</code> to work address in repo	PASSED
Age Cipher Roundtrip	<code>age --decrypt id_ed25519_dama.age</code>	Bit-for-bit identical to original private key	PASSED
Flake Flawlessness	<code>nix flake check --impure</code>	All 17 outputs, derivations & configs pass	PASSED
Secrets Upstream	<code>jj git push --remote gh</code>	Pushed rev <code>4f3f7e39</code> to <code>smunix/nix-secrets</code>	PASSED
GitLab Work Auth	<code>git@gitlab.com:damacs1/docs.git</code>	Authenticates as <code>@psalumu-dama</code> via <code>gitlab.com-dama</code>	PASSED

Table 2: Empirical Test Matrix across SSH, Git, Jujutsu, and Age Cryptography

Exhaustive Failure Modes & Invariant Leaks

- SSH Agent Exhaustion (Too many authentication failures):** When using GNOME Keyring / GCR SSH agent, multiple keys accumulate in memory. If `IdentitiesOnly yes` is omitted, the client offers keys in order of arrival. GitHub rejects connections after 6 offerings. Explicit `IdentitiesOnly yes` in both host stanzas completely prevents this exhaustion.
- Unintentional Personal Email Leakage on Work Commits:** If a developer initializes a work repo outside `~/Projects/dama/` without directory scoping, Git falls back to the default `Providence.Salumu@smunix.com`. To mitigate this, both directory scoping (`~/Projects/dama/` and `~/Projects/work/`) and remote URL rewriting (`insteadOf`) are deployed simultaneously.
- Plaintext SSH Keys in World-Readable Nix Store:** Copying private keys into `nix-config` or committing them unencrypted causes Nix to materialize them into `/nix/store/` with mode `0444` (readable by every process on the host). All keys committed to `nix-secrets` are strictly encrypted via `age -R .age-recipients`.
- OpenSSH Config-File Identity Accumulation & Precedence:** Specifying `core.sshCommand = "ssh -i ~/.ssh/id_ed25519_dama"` is insufficient if the remote host remains `github.com`. In OpenSSH, command-line `-i` flags append to, rather

than override, the `IdentityFile` directives declared in `~/.ssh/config` under `Host github.com`. OpenSSH presents the personal key `id_ed25519` first, GitHub recognizes and accepts it as `smunix`, and subsequent accesses to private work repositories fail with `ERROR: Repository not found`. Transport isolation requires explicitly rerouting via `Host github.com-dama` or scoping `url."git@github.com-dama:".insteadOf = "git@github.com:"` within the work directory tree.

Architectural Resilience & Graceful Fallback

The architecture provides complete offline and multi-tier resilience:

- **Offline Execution:** Local operations on `smunix` do not depend on network access to `nix-secrets` or GitHub; keys are stored locally under `~/.ssh/` with mode `0600`.
- **Key Recovery & Machine Migration:** If the machine is reprovisioned or restored from scratch, the developer decrypts `hosts/smunix/ssh/id_ed25519.age` and `id_ed25519_dama.age` from `nix-secrets` using their hardware YubiKey / `admin_key1`, restoring the full development workstation state seamlessly.
- **Lock Synchronization:** The `nix-config` flake lock explicitly references the latest secrets commit (`4f3f7e39`), maintaining strict reproducible provenance.

Upstream Grounding & Landmark References

- [1] OpenSSH Development Team. *OpenSSH Client Configuration (ssh_config(5)) Manual*. OpenBSD Project, 2024. https://man.openbsd.org/ssh_config
- [2] Git Development Community. *Git Configuration & Conditional Includes Documentation*. Git SCM Project, 2026. https://git-scm.com/docs/git-config#_conditional_includes
- [3] Jujutsu Team. *Jujutsu VCS Conditional Configuration Scopes*. Jujutsu Documentation, 2026. <https://docs.jj-vcs.dev/latest/config/#conditional-variables>
- [4] Filippo Valsorda. *The age File Encryption Format & Tool Specification*. age-encryption.org, 2024. <https://age-encryption.org/v1>